

< CodeResMark />

Technical Design Document

A Portfolio-Grade Browser Extension for Intelligent Bookmark Management

Version
1.0.0

Stack
Node · Mongo · Redis

Type
Portfolio Project

Table of Contents

Table of Contents.....

1

1.1 Folder Structure.....

1

1.2 Data Flow: Request → Database

2

2.1 Schema Design & Normalization

3

2.2 Indexing Strategy

3

2.3 Concurrent Write Strategy

3

4.1 Centralized Error Handler

4

4.2 Logging with Winston + Morgan

5

4.3 Log Levels & What to Track

5

5.1 Authentication Flow

5

5.2 Security Checklist.....

5

6.1 Most Expensive Operation: AI Classification

6

6.2 Optimization Architecture

6

Appendix: Environment Variables

8

01 System Architecture

1.1 Folder Structure

The project follows a strict Controllers → Services → Models (CSM) layered architecture. Each layer has a single responsibility, making the codebase easy to test and extend.

```
coderesmark-api/  
├── src/
```

```
├── config/                # env, db, redis, logger setup
├── controllers/           # HTTP layer - parse req, call service, return res
│   ├── auth.controller.js
│   ├── bookmark.controller.js
│   └── category.controller.js
├── services/              # Business logic - all decisions live here
│   ├── auth.service.js
│   ├── bookmark.service.js
│   ├── classifier.service.js # AI integration
│   └── category.service.js
├── models/                # Mongoose schemas + static methods
│   ├── user.model.js
│   ├── bookmark.model.js
│   └── category.model.js
├── workers/               # BullMQ job processors (async classification)
│   └── classifier.worker.js
├── queues/                # Queue definitions and job producers
│   └── classify.queue.js
├── middleware/            # Auth, error handler, rate limiter, sanitizer
├── routes/                # Express router definitions only
├── utils/                 # Pure helpers (hashing, url normalizer, etc.)
├── app.js                 # Express app factory
├── tests/
│   ├── unit/              # Service + util tests (Jest)
│   └── integration/       # Supertest API endpoint tests
├── extension/             # Chrome extension source
│   ├── background/        # Service worker (offline queue, auth)
│   ├── popup/             # React popup UI
│   └── content/           # Content script (bookmark interceptor)
└── docker-compose.yml     # MongoDB + Redis local dev stack
```

1.2 Data Flow: Request → Database

Step	Layer	Responsibility
1	Extension	User bookmarks a URL → content script fires <code>chrome.runtime.sendMessage</code>
2	Background SW	Queues request locally (offline resilience), sends POST <code>/bookmarks</code> with Bearer token
3	Middleware	JWT verified → <code>userId</code> attached to req. Input sanitized by <code>express-mongo-sanitize</code>
4	Controller	Extracts & validates body fields. Delegates to <code>BookmarkService</code> — zero business logic here
5	Service	Normalises URL, computes <code>urlHash</code> , calls <code>Model.findOneAndUpdate (upsert)</code> . Returns saved doc
6	Queue	Enqueues classify job in Redis/BullMQ. Returns 202 Accepted immediately to client
7	Worker	Async: fetches meta tags, calls AI API, patches <code>bookmark.aiMetadata</code> , sets status: 'classified'
8	SSE / Poll	Extension receives classification update via Server-Sent Events, updates popup UI

02 Database Strategy

2.1 Schema Design & Normalization

The schema is intentionally semi-denormalized. Category names are not embedded in bookmarks — instead a categoryId reference is stored. This avoids fan-out update problems: renaming a category requires one write, not thousands.

Collection	Key Fields	Relationships	Validation
users	_id, email, passwordHash, settings	1:many → bookmarks	email unique, required; passwordHash never returned in queries
bookmarks	_id, userId, urlHash, url, title, category, tags[], aiMetadata, status	many:1 → users, many:1 → categories	urlHash unique compound; status enum; confidence 0–1 range validator
categories	_id, userId, name, color, isSystem	1:many → bookmarks	isSystem prevents deletion of 'Uncategorized'; color hex pattern match

2.2 Indexing Strategy

- **Compound index** { userId: 1, status: 1 } — primary popup fetch; covers the 90% case
- **Unique index** { urlHash: 1 } — enforces deduplication at the DB layer regardless of race conditions
- **Text index** { title, 'aiMetadata.summary', tags } — powers full-text search across all bookmark metadata
- **TTL index** { createdAt: 1 } on a failed-jobs collection — auto-cleans stale classifier errors after 72 hours
- **Partial index** { userId: 1, category: 1 } where status='classified' — fast category-filter view skips pending records

2.3 Concurrent Write Strategy

Two patterns protect against data corruption under concurrent writes:

- **Atomic upsert:** Model.findOneAndUpdate({ urlHash }, update, { upsert: true, new: true }) eliminates TOCTOU race on duplicate bookmarks.
- **Optimistic locking:** A __v (version key) field on Bookmark documents uses Mongoose's built-in OCC. Conflicting concurrent updates will throw a VersionError, which the service retries once.
- **Queue serialisation:** All AI classification writes go through BullMQ with concurrency: 1 per bookmark ID. Workers cannot double-write aiMetadata.

03 Tech Stack Rationale

Layer	Technology	Alternatives Considered	Why This Choice
Runtime	Node.js 20 LTS	Deno, Bun	Mature ecosystem; non-blocking I/O ideal for high-frequency extension events;

Layer	Technology	Alternatives Considered	Why This Choice
			first-class Chrome extension tooling
Framework	Express 5	Fastify, Hono	Universal familiarity for portfolio reviewers; middleware ecosystem; straightforward error-handler pattern
Database	MongoDB Atlas	PostgreSQL	Schema flexibility for dynamic AI-generated metadata; native vector search for semantic bookmarks (Phase 3)
ODM	Mongoose 8	Prisma (MongoDB), raw driver	Schema validation, middleware hooks, and populate() cover all relational needs without ORM overhead
Cache / Queue	Redis 7 + BullMQ	RabbitMQ, SQS	Dual-purpose: token storage AND job queue in one service. BullMQ's retry/backoff handles flaky AI API perfectly
Auth	jsonwebtoken + bcrypt	Passport.js, Auth0	No third-party dependency; demonstrates understanding of the auth primitives. Bcrypt cost-12 is the industry standard
Extension UI	React 18 + Vite	Vanilla JS, Svelte	Component reusability for category cards; fast HMR during dev; familiar to every engineering reviewer
Testing	Jest + Supertest	Vitest, Mocha	De facto standard; code-coverage built in; Supertest mounts Express without a live server — fast CI

04 Error Handling & Logging

4.1 Centralized Error Handler

All errors are funneled through a single Express error-handling middleware. Services throw typed errors; controllers never catch — they let errors propagate.

```
// utils/AppError.js
class AppError extends Error {
  constructor(message, statusCode, code) {
    super(message);
    this.statusCode = statusCode;
    this.code = code; // e.g. 'BOOKMARK_DUPLICATE'
    this.isOperational = true; // Distinguishes from programmer errors
  }
}

// middleware/errorHandler.js
module.exports = (err, req, res, next) => {
  const status = err.isOperational ? err.statusCode : 500;
```

```

logger.error({ err, req: { method: req.method, url: req.url } });
res.status(status).json({
  success: false,
  error: { code: err.code || 'INTERNAL_ERROR', message: err.message }
});
};

```

4.2 Logging with Winston + Morgan

Tool	Purpose	Configuration
Winston	Application / error logs	3 transports: Console (dev, colored), File (errors.log), File (combined.log). JSON format in production for log aggregator ingestion (Datadog / Logtail)
Morgan	HTTP access logs	Streams to Winston. 'combined' format in prod; 'dev' in local. Skips health-check routes to reduce noise
BullMQ events	Queue / worker telemetry	completed, failed, stalled events logged with jobId, bookmarkId, duration. Failed jobs log full error stack for AI API debugging

4.3 Log Levels & What to Track

- **error**: Unhandled exceptions, DB write failures, AI API 5xx responses
- **warn**: JWT expiry on valid refresh, rate-limit hits, low AI confidence scores (<0.4)
- **info**: Successful auth events, bookmark saved, classification complete
- **debug**: SQL/Mongo query timing, Redis cache hit/miss, full request/response body (dev only)

05 Security & Privacy

5.1 Authentication Flow

```

REGISTER → bcrypt.hash(password, 12) → store passwordHash only
LOGIN    → bcrypt.compare()           → issue accessToken (JWT, 15 min)
                                           → issue refreshToken (opaque UUID, 7d)
                                           → stored as HttpOnly + Secure cookie
API CALL → Authorization: Bearer <accessToken>
           → verifyJWT() → attach userId to req
REFRESH  → POST /auth/refresh with cookie
           → validate refreshToken against Redis allowlist
           → rotate BOTH tokens (old refresh deleted from Redis)
LOGOUT   → DELETE refreshToken from Redis → cookie cleared

```

5.2 Security Checklist

✓	Control	Implementation Detail
✓	NoSQL Injection	express-mongo-sanitize strips \$ and . from all req.body / req.query fields before any controller runs
✓	XSS	helmet() sets Content-Security-Policy. DOMPurify used in extension popup before rendering any AI-generated summary strings
✓	CSRF	API is stateless (JWT Bearer). Extension origin checked via cors() with explicit allowlist. RefreshToken rotates on every use
✓	Brute Force	express-rate-limit: 10 req/15min on /auth/*. Redis stores counters — consistent across horizontal pod replicas
✓	Password Storage	bcrypt cost factor 12. Plain-text password never logged. select: false on passwordHash field in User model
✓	Secrets Management	API keys (OpenAI etc.) in .env, never bundled in extension. dotenv-safe throws on missing keys at startup
✓	URL Validation	All bookmark URLs validated via new URL() + blocklist rejecting file://, javascript:, chrome-extension:// schemes
✓	HTTPS Only	Helmet HSTS header. Extension manifest enforces https:// host_permissions. All cookies use Secure flag
✓	Least Privilege	Extension requests only tabs and storage permissions in manifest. Content scripts run in isolated world
✓	Token Leakage	accessToken never stored in localStorage. Stored in memory (React state) in popup. Expires in 15 min

06 Efficiency & Optimization

6.1 Most Expensive Operation: AI Classification

The single most expensive operation is the AI metadata extraction call. A naive synchronous implementation would block every POST /bookmarks request for 2–5 seconds and fail completely if the AI API is rate-limited or down.

6.2 Optimization Architecture

Strategy	Tool	Outcome
Async Job Queue	BullMQ + Redis	POST /bookmarks returns 202 in <50ms. AI work happens in a background worker on a separate process — API never blocks
Retry with Backoff	BullMQ options	Failed jobs retry 3x with exponential backoff (1s, 5s, 30s). Handles transient AI API rate limits gracefully
Result Caching	Redis (TTL 7d)	If two users bookmark the same URL, the second classification is served from cache. Key: SHA-256(normalizedUrl)

Strategy	Tool	Outcome
Lightweight Pre-filter	og:meta scraper	Worker reads og:description and meta tags first. If confidence > 0.8 from meta alone, skips expensive full-page AI call
Popup Performance	Redis + pagination	GET /bookmarks is served from a Redis-cached user bookmark list (60s TTL). Cache invalidated on any write
Compression	compression middleware	Gzip applied to all API responses. Bookmark list payloads shrink ~70%, critical for extension popup load time

07 Implementation Phases

Each sprint maps to a Git branch pattern: feature/sprint-1-*, feature/sprint-2-*, etc. Merge to main only when all sprint tests pass.

Sprint 1 MVP — Foundation *[Branch: sprint-1/mvp]*

- ▶ Initialize monorepo: /api, /extension, /tests with ESLint + Prettier config
- ▶ Docker Compose: MongoDB + Redis services for local dev
- ▶ User model + auth routes: POST /register, POST /login with bcrypt + JWT
- ▶ Bookmark model with urlHash deduplication (findOneAndUpdate upsert)
- ▶ Category model with isSystem 'Uncategorized' seed
- ▶ Basic CRUD endpoints: GET/POST/PATCH/DELETE /bookmarks, GET/POST /categories
- ▶ Chrome extension manifest v3: background service worker, popup skeleton
- ▶ Content script: intercepts bookmark creation via chrome.bookmarks.onCreated
- ▶ Popup: hardcoded static UI rendering bookmark list from API
- ▶ Git milestone: Working bookmark save + list with auth — no AI yet

Sprint 2 Core Logic — AI & Queue *[Branch: sprint-2/core-logic]*

- ▶ BullMQ classify queue connected to Redis — job producer in bookmark.service.js
- ▶ Classifier worker: og:meta scrape → OpenAI API call → patch aiMetadata
- ▶ Confidence-based fallback: score < 0.4 sets status='needs-review'
- ▶ Server-Sent Events endpoint: GET /bookmarks/updates (streams status changes to extension)
- ▶ Extension popup: React components — CategoryCard, BookmarkItem, SearchBar
- ▶ Popup updates in real-time when classification completes via SSE listener
- ▶ Redis result cache for shared URLs (TTL 7 days)
- ▶ Fuzzy category deduplication on POST /categories (Levenshtein distance check)
- ▶ Offline queue in extension service worker — drains on reconnect with upsert
- ▶ Git milestone: Full AI classification pipeline end-to-end working

Sprint 3 Security & Testing *[Branch: sprint-3/security-testing]*

- ▶ Refresh token rotation: Redis allowlist, HttpOnly cookie, token invalidation on logout
- ▶ express-mongo-sanitize + helmet() + cors() configured on all routes
- ▶ Rate limiting on /auth/* (10 req/15min, Redis-backed for horizontal scale)
- ▶ URL blocklist validation (file://, javascript:// schemes rejected)
- ▶ DOMPurify sanitization on all AI-generated strings rendered in popup
- ▶ Jest unit tests: auth.service, bookmark.service, classifier.service (mock AI API)
- ▶ Supertest integration tests: full CRUD flows, auth edge cases, duplicate handling
- ▶ BullMQ worker tests: retry logic, cache hit/miss, low-confidence fallback
- ▶ Code coverage target: 80%+ on all service files
- ▶ Git milestone: All tests green, security audit passed, coverage badge on README

Sprint 4 Polish & Portfolio Presentation *[Branch: sprint-4/polish]*

- ▶ Vector embedding search: generate embeddings on classify, store in MongoDB Atlas Vector Search
- ▶ Semantic search endpoint: GET /bookmarks/semantic?q= using cosine ANN
- ▶ Winston structured logging + Morgan HTTP logs piped to combined.log
- ▶ compression middleware + response time header on all routes
- ▶ Extension popup polish: skeleton loading states, category color pickers, smooth animations
- ▶ README: architecture diagram, setup guide, API contract, test instructions
- ▶ GitHub Actions CI: lint → test → coverage report on every PR
- ▶ Docker production Dockerfile with multi-stage build (dev deps excluded)
- ▶ Deploy API to Railway or Render; extension submitted to Chrome Web Store (unlisted)
- ▶ Git milestone: Live demo link in README. Portfolio-ready with CI badge

Appendix: Environment Variables

```
# Server
PORT=3000
NODE_ENV=development

# MongoDB
MONGO_URI=mongodb://localhost:27017/coderesmark

# Redis
REDIS_URL=redis://localhost:6379

# JWT
JWT_ACCESS_SECRET=<256-bit random>
JWT_REFRESH_SECRET=<256-bit random, different from access>
JWT_ACCESS_EXPIRY=15m
JWT_REFRESH_EXPIRY=7d
```

```
# AI
OPENAI_API_KEY=sk-...
OPENAI_MODEL=gpt-4o-mini
CLASSIFIER_CONFIDENCE_THRESHOLD=0.4

# Rate Limiting
RATE_LIMIT_WINDOW_MS=900000
RATE_LIMIT_MAX=10
```

All secrets must be stored in a .env file excluded from version control. Use dotenv-safe with a committed .env.example to enforce required variables at startup.

CodeResMark TDD · Built as a portfolio piece demonstrating production-grade backend engineering